CSCI 5302 HW5

April 21, 2026

Ram Prasad

[1]:
```
# standard imports
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from scipy.optimize import minimize
```

Q1

$$\min_{x,y} f(x,y) = e^{-x} + e^{-2y}$$

$$s.t.$$

$$\begin{aligned} x + y - 1 &\leq 0 \\ -x &\leq 0 \\ -y &\leq 0 \end{aligned}$$

Before applying the KKT conditions, I provide an alternate solution. Note that the objective function is strictly decreasing in both x and y, thus from any interior feasible point the objective can be further minimized by increasing either until hitting the boundary $x+y=1$. Thus $x+y=1$ at the minimzer, and substituting gives a 1D optimization problem,

$$\min_{0 \leq x \leq 1} \; f(x, 1-x) = e^{-x} + e^{-2(1-x)}.$$

Simplifying and differentiating,

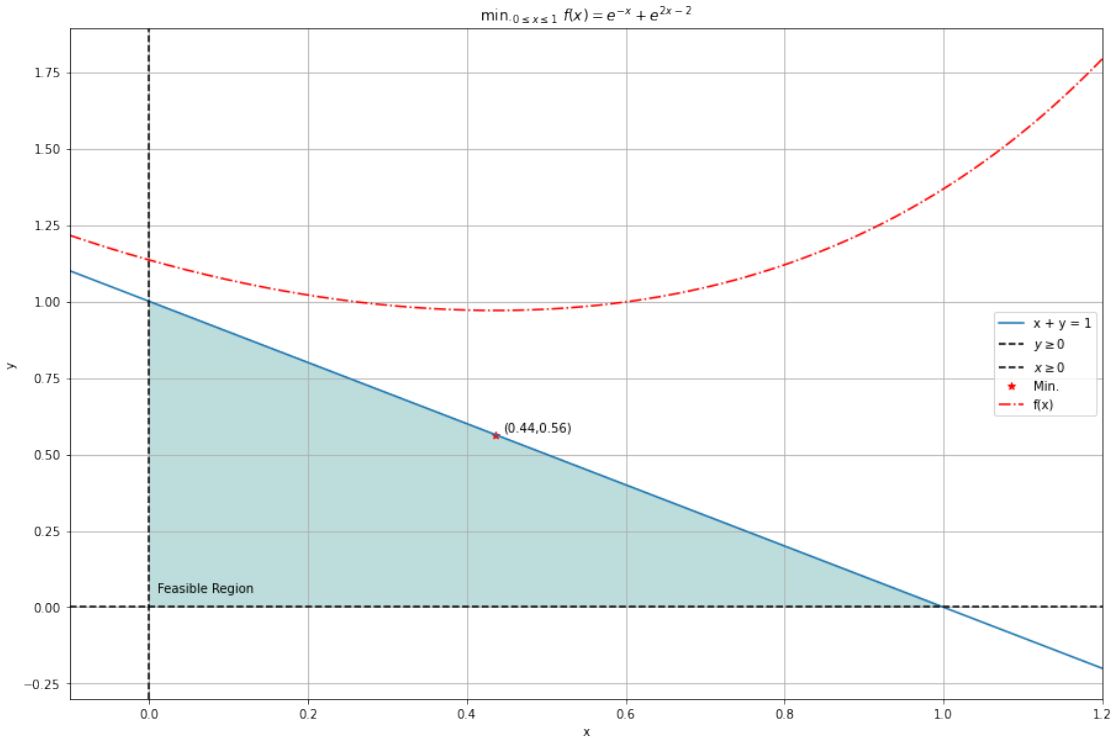
$$\begin{aligned} f(x) &= e^{-x} + e^{2x-2} \\ f'(x) &= -e^{-x} + 2e^{2x-2} \\ f'(x) = 0 &\implies -e^{-x} + 2e^{2x-2} = 0 \\ e^{-x} &= 2e^{2x-2} \\ 1 &= 2e^{3x-2} \\ e^{3x-2} &= \frac{1}{2} \\ 3x - 2 &= \ln\left(\tfrac{1}{2}\right) = -\ln 2. \end{aligned}$$

Thus

$$x^* = \frac{2 - \ln 2}{3}$$
$$y^* = 1 - x^* = \frac{1 + \ln 2}{3}.$$

Since $f''(x) = e^{-x} + 4e^{2x-2} > 0$ for all x, (x^*, y^*) is a global minimum on $[0, 1]$. I plot the feasible region and the 1D function below.

[2]:
```
# create grid
x = np.linspace(-0.1, 1.2, 1000)
y = np.linspace(-0.1, 1.2, 1000)
X, Y = np.meshgrid(x, y)

# feasible region mask
feasible = (X + Y <= 1) & (X >= 0) & (Y >= 0)

# plot
plt.figure(figsize=(15,10))

# shade feasible region
plt.contourf(X, Y, feasible, levels=[0.5, 1], alpha=0.3)

# plot constraint boundaries
plt.plot(x, 1 - x, label='x + y = 1')
plt.axhline(0, label=r"$y \geq 0$", linestyle="--", color="black")
plt.axvline(0, label=r"$x \geq 0$", linestyle="--", color="black")
plt.text(0.01, 0.05, "Feasible Region")

# scatter minimizer
x_star = (2 - np.log(2))/3
y_star = (1 + np.log(2))/3
plt.scatter(x_star, y_star, label="Min.", color="red", marker="*")
plt.text(x_star+0.01, y_star+0.01, s=f"({np.round(x_star,2)},{np.
 ↪round(y_star,2)})")

# plot 1D function
plt.plot(x, np.exp(-x) + np.exp(2*x-2), label="f(x)", linestyle="dashdot",␣
 ↪color="red")

# Labels, legend, display
plt.xlabel('x')
plt.ylabel('y')
plt.title(r'min.$_{0 \leq x \leq 1}$ $f(x)=e^{-x}+e^{2x-2}$')
plt.legend()
plt.grid()
plt.show()
```



I solve the optimization problem using KKT conditions.

a.

The gradient is

$$\boxed{\nabla f(x,y) = \begin{pmatrix} -e^{-x} \\ -2e^{-2y} \end{pmatrix}}.$$

The Hessian is

$$\boxed{H(x,y) = \begin{pmatrix} e^{-x} & 0 \\ 0 & 4e^{-2y} \end{pmatrix}}$$

with eigenvalues (taking diagonal elements)

$$\lambda_1 = e^{-x} > 0, \quad \lambda_2 = 4e^{-2y} > 0.$$

The Hessian is positive definite for all (x, y), thus f is strictly convex. The feasible set is defined by linear inequalities, hence it is convex and this is a convex optimization problem.

b.

I assume only the constraint $h_1(x, y) = x+y-1 \le 0$ is binding (strict equality holds), thus $y = 1-x$.

Using the stationarity condition,

$$\begin{aligned}\nabla f &= \lambda \nabla h_1, \\ \begin{pmatrix} -e^{-x} \\ -2e^{-2y} \end{pmatrix} &= \lambda \begin{pmatrix} 1 \\ 1 \end{pmatrix} \implies e^{-x} = 2e^{-2y}\end{aligned}$$

Substituting $y = 1 - x$,

$$\begin{aligned} e^{-x} &= 2e^{-2(1-x)} \\ &= 2e^{2x-2} \implies 1 = 2e^{3x-2} \\ e^{3x-2} &= \frac{1}{2}. \end{aligned}$$

Taking logs,

$$\begin{aligned} 3x - 2 = \ln\left(\frac{1}{2}\right) &= -\ln 2 \\ &\implies \boxed{x = \frac{2 - \ln 2}{3}} \\ &\implies \boxed{y = 1 - x = \frac{1 + \ln 2}{3}}. \end{aligned}$$

c

I form the Lagrangian,

$$\mathcal{L}(x, y, \lambda_1, \lambda_2, \lambda_3) = e^{-x} + e^{-2y} + \lambda_1(x + y - 1) + \lambda_2(-x) + \lambda_3(-y).$$

Condition 1 implies

$$\begin{aligned} \frac{\partial \mathcal{L}}{\partial x} &= -e^{-x} + \lambda_1 - \lambda_2 = 0 \implies e^{-x} = \lambda_1 - \lambda_2, \\ \frac{\partial \mathcal{L}}{\partial y} &= -2e^{-2y} + \lambda_1 - \lambda_3 = 0 \implies 2e^{-2y} = \lambda_1 - \lambda_3. \end{aligned}$$

There are no equality constraints.

Condition 3 implies

$$x + y - 1 \leq 0, \quad x \geq 0, \quad y \geq 0.$$

Condition 4 implies

$$\lambda_1, \lambda_2, \lambda_3 \geq 0.$$

Condition 5 implies

$$\begin{aligned} \lambda_1(x + y - 1) &= 0 \\ \lambda_2(-x) &= 0 \\ \lambda_3(-y) &= 0. \end{aligned}$$

As stated above, the solution must lie on $x + y = 1$, thus the constraint is active and by complementarity, $\lambda_1 \neq 0$. If $x \geq 0$ is binding, the only solution is $(0, 1)$. Similarly, if $y \geq 0$ is binding, the only solution is $(1, 0)$, so I evaluate the objective at each,

$$\begin{aligned} f(0,1) &= 1 + \frac{1}{e^2} \approx 1.135 \\ f(1,0) &= \frac{1}{e} + 1 \approx 1.368 \\ f(0.5, 0.5) &= \frac{1}{e^{0.5}} + \frac{1}{e} \approx 0.9744. \end{aligned}$$

Neither $(0, 1)$ nor $(1, 0)$ is a minimizer, so neither constraint is binding and thus

$$\lambda_2 = 0, \quad \lambda_3 = 0.$$

Solving the system from condition 1 gives

$$\begin{aligned} \lambda_1 &= e^{-x} = 2e^{-2y} \\ x + y &= 1 \\ e^{-x} &= 2e^{-2(1-x)} \\ -x &= \ln(2) - 2 + 2x \\ 3x &= 2 - \ln(2) \\ x^* &= \frac{2 - \ln(2)}{3} \\ y^* &= \frac{1 + \ln(2)}{3} \\ \lambda_1^* &= e^{-\frac{2-\ln 2}{3}} > 0 \\ \lambda_2^*, \lambda_3^* &= 0. \end{aligned}$$

All KKT conditions hold. The Lagrange multipliers are non-negative, the inequality constraints hold (the first binds, the second and third do not), the gradient of the Lagrangian at (x^*, y^*) is 0, and complementarity holds as h_1 is 0, and λ_2 and λ_3 are zero.

$$\boxed{x^* = \frac{2 - \ln(2)}{3} \approx 0.4356}$$

$$\boxed{y^* = \frac{1 + \ln(2)}{3} \approx 0.5644}$$

$$\boxed{\lambda_1^* = e^{-\frac{2-\ln 2}{3}} \approx 0.6468 > 0}$$

$$\boxed{\lambda_2^*, \lambda_3^* = 0.}$$

I check my solution using scipy.

[3]:
```
# objective function
def f(v): return np.exp(-v[0]) + np.exp(-2*v[1])
```

```python
# initial guess in feasible region
x0 = np.array([0.4, 0.4])

# constraints
constraints = [{'type': 'ineq', 'fun': lambda v: 1 - (v[0] + v[1])}, # x + y <= 1
               {'type': 'ineq', 'fun': lambda v: v[0]},    # x >= 0
               {'type': 'ineq', 'fun': lambda v: v[1]}] # y >= 0

# solve and check output
result = minimize(f, x0, method='SLSQP', constraints=constraints, tol=1e-10)
x_star, y_star = result.x
print("=== (x*,y*): ===")
print((x_star, y_star))
print("\n=== f(x*,y*): ===")
print(result.fun)
print("\n=== Lagrange Multipliers: ===")
print("Lambda1*=", np.exp(-x_star))
print("Lambda2*=", 0)
print("Lambda3*=", 0)
```

```
=== (x*,y*): ===
(0.4356162157586015, 0.5643837842413989)

=== f(x*,y*): ===
0.970297553470193

=== Lagrange Multipliers: ===
Lambda1*= 0.6468659352552287
Lambda2*= 0
Lambda3*= 0
```

Q2

A sine wave of frequency k can be represented as

$$\sin(2\pi kt) = i\left[\frac{e^{-2\pi ikt} - e^{2\pi ikt}}{2}\right].$$

a.

For $k = 2\text{Hz}$, the sine wave can be represented as

$$\boxed{\sin(4\pi t)}.$$

b.

Let s be the number of samples taken per second. Sampling means evaluating the function

$$\sin(4\pi\frac{n}{s})$$

at $n = 0, 1, 2, 3, \ldots$. For example, when $s = 2$, sample at

$$0, 1/2, 1, 3/2, \ldots$$

and when $s = 3$, sample at

$$0, 1/3, 2/3, 1, \ldots$$

At $s = 1$, the function takes values

$$\sin(0), \sin(4\pi), \sin(8\pi), \sin(12\pi), \ldots$$

At each multiple of 4π, the sin function is 0, thus $\boxed{\text{all sampled values are 0}}$.

c.

The Nyquist Shannon sampling theorem states that a function with frequency k can be completely determined from its sampled values by a sequence of points spaced less than $1/2k$ seconds apart, or equivalently the sampling rate must be at least 2k. In this case, 4 samples per second gives all 0s, so identifying the frequency requires sampling $\boxed{\text{5 or more times per second}}$.

[4]:
```
# hold results
sampling_freq = []

# evaluate function at different sampling frequencies
for s in range(1,10+1):
    sampling_freq.append([s] + [np.sin(4*np.pi*(n/s))  for n in range(10+1)])

# convert to dataframe
sampling_freq = pd.DataFrame(sampling_freq)

# set columns and display
sampling_freq.columns = ["S"] + [f"n={n-1}" for n in sampling_freq.columns[1:]]
sampling_freq.set_index("S", inplace=True)
display(sampling_freq.round(4))
```

```
    n=0     n=1     n=2     n=3     n=4     n=5     n=6     n=7     n=8  \
S
1   0.0 -0.0000 -0.0000 -0.0000 -0.0000 -0.0000 -0.0000 -0.0000 -0.0000
2   0.0 -0.0000 -0.0000 -0.0000 -0.0000 -0.0000 -0.0000 -0.0000 -0.0000
3   0.0 -0.8660  0.8660 -0.0000 -0.8660  0.8660 -0.0000 -0.8660  0.8660
4   0.0  0.0000 -0.0000  0.0000 -0.0000  0.0000 -0.0000  0.0000 -0.0000
5   0.0  0.5878 -0.9511  0.9511 -0.5878 -0.0000  0.5878 -0.9511  0.9511
6   0.0  0.8660 -0.8660 -0.0000  0.8660 -0.8660 -0.0000  0.8660 -0.8660
7   0.0  0.9749 -0.4339 -0.7818  0.7818  0.4339 -0.9749 -0.0000  0.9749
8   0.0  1.0000  0.0000 -1.0000 -0.0000  1.0000  0.0000 -1.0000 -0.0000
9   0.0  0.9848  0.3420 -0.8660 -0.6428  0.6428  0.8660 -0.3420 -0.9848
10  0.0  0.9511  0.5878 -0.5878 -0.9511 -0.0000  0.9511  0.5878 -0.5878

       n=9     n=10
S
```

```
1  -0.0000 -0.0000
2  -0.0000 -0.0000
3  -0.0000 -0.8660
4   0.0000 -0.0000
5  -0.5878 -0.0000
6  -0.0000  0.8660
7  -0.4339 -0.7818
8   1.0000  0.0000
9  -0.0000  0.9848
10 -0.9511 -0.0000
```

Q3

Using the nth root of unity,

$$\begin{aligned}\omega_n &= \cos\left(\frac{2\pi}{n}\right) - i \cdot \sin\left(\frac{2\pi}{n}\right) \\ &= \cos\left(-\frac{2\pi}{n}\right) + i \cdot \sin\left(-\frac{2\pi}{n}\right),\end{aligned}$$

for $n = 360$, the angle is $\boxed{\theta = -\frac{\pi}{180}\text{radians or } -1 \text{ degree}}$.

Q4-8

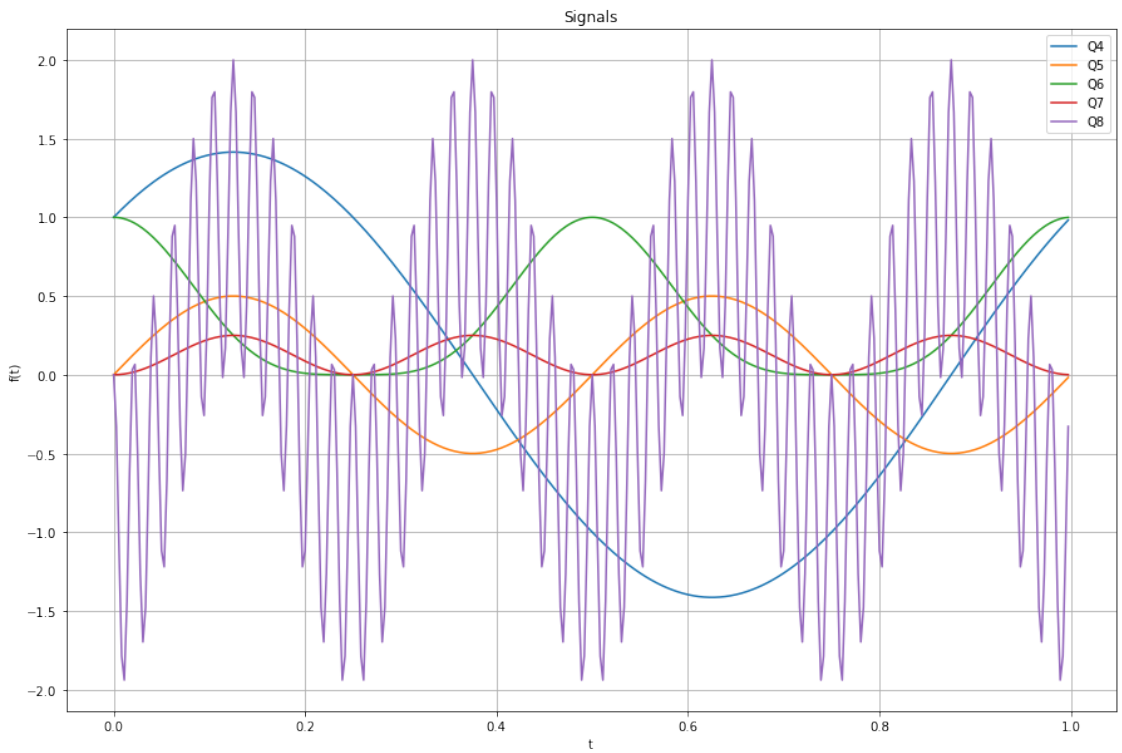
I sample the given signals at 360 equally spaced points in $[0, 1)$, compute the DFT and identify the non-zero entries using tolerances for the real and complex components separately, and plot the signals.

[5]:
```
# define signals
signals = {"Q4":lambda t: np.cos(2*np.pi*t) + np.sin(2*np.pi*t),
           "Q5":lambda t: np.cos(2*np.pi*t) * np.sin(2*np.pi*t),
           "Q6":lambda t: np.cos(2*np.pi*t)**4,
           "Q7":lambda t: np.cos(2*np.pi*t)**2 - np.cos(2*np.pi*t)**4,
           "Q8":lambda t: np.cos(48*2*np.pi*t) - np.cos(4*2*np.pi*t)}


# define sample points on interval
interval = np.linspace(0,1,360, endpoint=False)
assert(len(interval)==360)


# loop through signals
plt.figure(figsize=(15,10))
for lab, f in signals.items():

    # check signal is periodic
    try:
```

```
        assert(np.isclose(f(0), f(1), atol=1e-08))
    except AssertionError:
        print("Warning: Non-periodic signal, f(0)=/=f(1):")
        print(f(0), f(1))

    # sample function on interval
    sample_values = f(interval)

    # plot signal
    plt.plot(interval, sample_values, label=lab)

    # compute DFT using FFT
    dft = np.fft.fft(sample_values)

    # capture indices and values of non-zero coefficients
    nonzero_freq = [(idx, val) for idx, val in enumerate(dft) if np.abs(val.
↪real) >= 1e-5 or np.abs(val.imag) >= 1e-5]
    print(f"\n=== Signal: {lab} ===")
    for pair in nonzero_freq:
        print(f"Index: {pair[0]}, Coefficient: {pair[1]}")

# grid, label, legend
plt.legend()
plt.grid()
plt.xlabel("t")
plt.ylabel("f(t)")
plt.title("Signals")
plt.show()
```

```
=== Signal: Q4 ===
Index: 1, Coefficient: (180-180j)
Index: 359, Coefficient: (179.99999999999997+180j)

=== Signal: Q5 ===
Index: 2, Coefficient: (9.509958019981676e-16-90j)
Index: 358, Coefficient: (-3.4200747082132177e-15+90j)

=== Signal: Q6 ===
Index: 0, Coefficient: (135+0j)
Index: 2, Coefficient: (90+7.494005416219807e-16j)
Index: 4, Coefficient: (22.5+9.992007221626409e-16j)
Index: 356, Coefficient: (22.5+1.6838478796805535e-16j)
Index: 358, Coefficient: (89.99999999999999-2.6597803274274875e-15j)

=== Signal: Q7 ===
Index: 0, Coefficient: (45+0j)
Index: 4, Coefficient: (-22.5+2.220446049250313e-16j)
```

```
Index: 356, Coefficient: (-22.5-1.440999895357232e-15j)

=== Signal: Q8 ===
Index: 4, Coefficient: (-179.99999999999991+2.886579864025407e-14j)
Index: 48, Coefficient: (179.99999999999994-2.7612096847313602e-14j)
Index: 312, Coefficient: (179.99999999999994+3.32460996989003e-14j)
Index: 356, Coefficient: (-179.9999999999999-3.506258469822175e-14j)
```



The signals are real valued functions, thus I examine non-zero (based on real or imaginary component) coefficients of the DFT for each signal to identify frequencies present in the signal. Nonzero DFT coefficients come in conjugate pairs corresponding to the frequency and negative frequency. The results above show $\boxed{\text{1 for Q4, 2 for Q5, 0, 2, and 4 for Q6, 0 and 4 for Q7, and 4 and 48 for Q8}}$. These are the frequencies present, the print output shows indices for pairs k and $360 - k$.

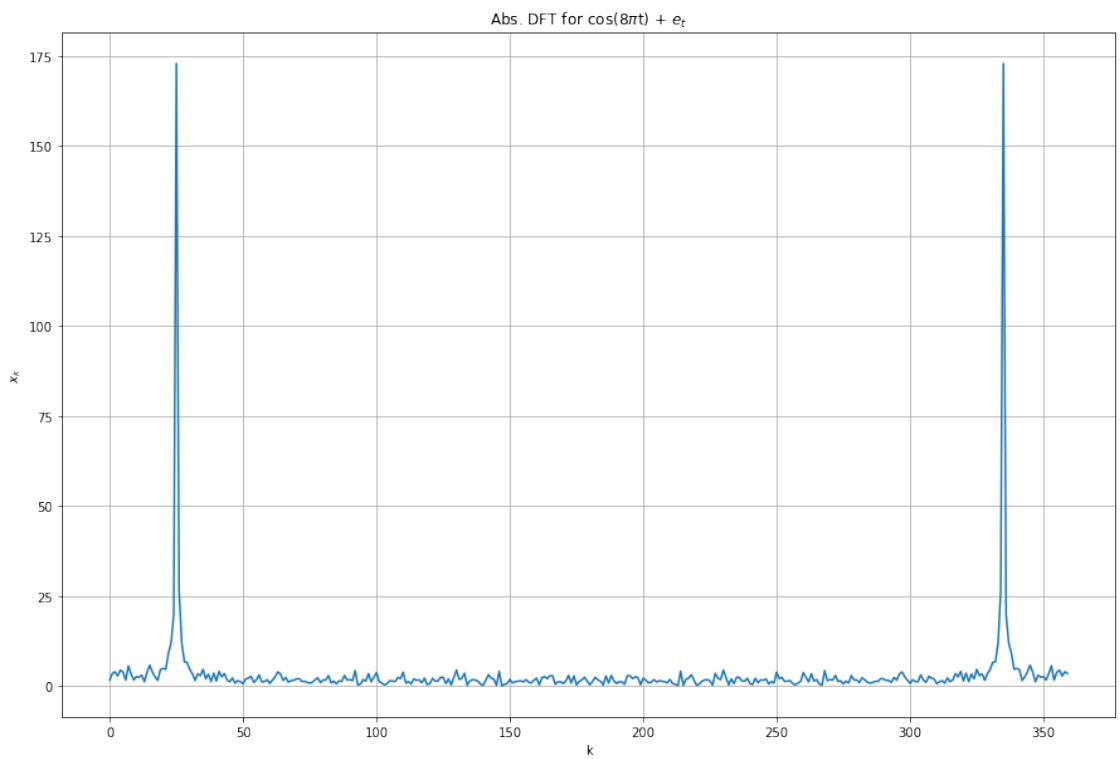
Q9-11

I compute the DFT of a cosine wave with white noise (mean zero, variance 0.01),

$$f(t) = \cos(8\pi t) + e_t, \quad e_t \sim \text{i.i.d}(0, 0.01).$$

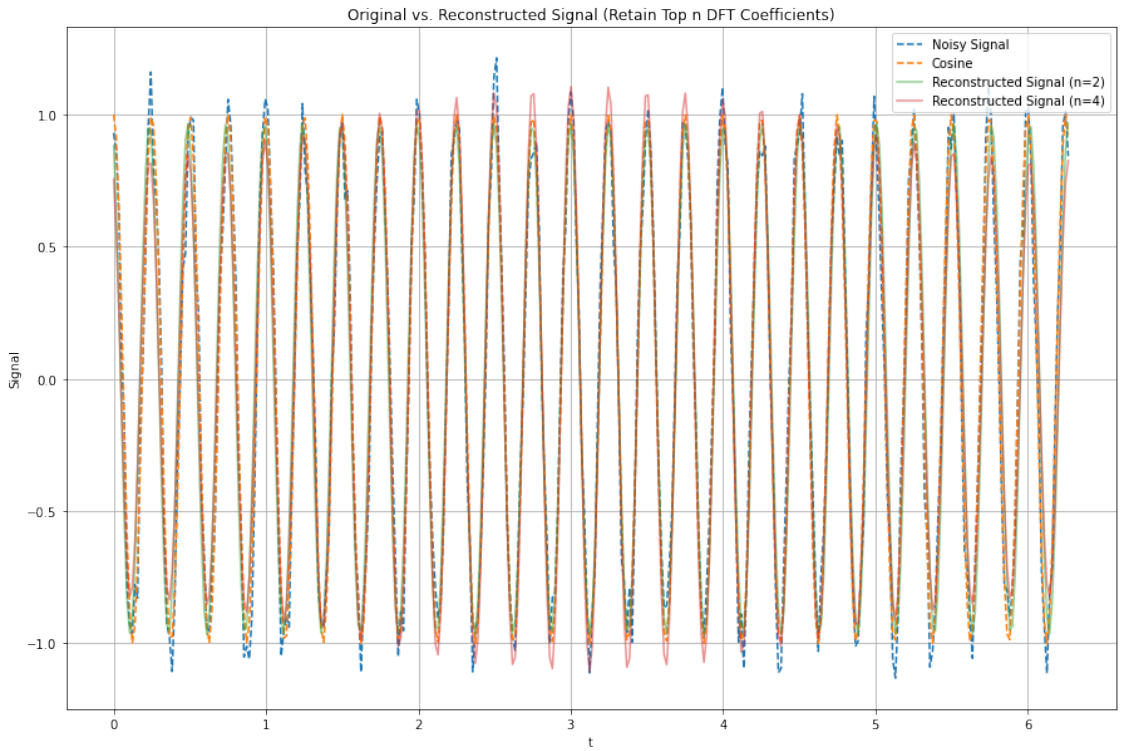
[6]:
```
# set interval
interval_q9 = np.linspace(0,2*np.pi,360, endpoint=False)
assert(len(interval_q9)==360)

# sample cosine plus noise signal
np.random.seed(1337)
cos_sample_values = np.cos(8*np.pi*interval_q9) + np.random.randn(360)/10

# compute DFT using FFT
cos_dft = np.fft.fft(cos_sample_values)

# plot DFT
plt.figure(figsize=(15,10))
plt.plot(np.abs(cos_dft))
plt.title(r"Abs. DFT for cos(8$\pi$t) + $e_t$")
plt.ylabel(r"$x_k$")
plt.xlabel("k")
plt.grid()
plt.show()

# identify four entries with largest absolute value
sorted_cos_dft = sorted([(idx, np.abs(val)) for idx, val in enumerate(cos_dft)],␣
↪key=lambda x: x[1])[::-1]
print("=== Largest 8 DFT entries by absolute value: ===")
for pair in sorted_cos_dft[0:8]:
    print(pair)

# setup plot against original
plt.figure(figsize=(15,10))
plt.plot(interval_q9, cos_sample_values, label="Noisy Signal", linestyle="--")
plt.plot(interval_q9, np.cos(8*np.pi*interval_q9), label="Cosine",␣
↪linestyle="--")

# zero out all but largest 2 and 4 entries
for n in [2,4]:

    # indices of largest n entries
    keep_idx = np.array(np.abs(cos_dft)).argsort()[-n:]

    # set DFT coefficients to zero
    zero_cos_dft = []
    for idx, val in enumerate(cos_dft):
        if idx not in keep_idx:
            zero_cos_dft.append(0.0)
        else:
            zero_cos_dft.append(val)
```

```
    # take inverse DFT
    zero_cos = np.fft.ifft(zero_cos_dft).real

    # plot reconstructed signal
    plt.plot(interval_q9, zero_cos, label=f"Reconstructed Signal (n={n})",␣
↪alpha=0.5)

# title, labels, grid, display
plt.title(f"Original vs. Reconstructed Signal (Top n DFT Retained)")
plt.ylabel(r"Signal")
plt.xlabel("t")
plt.grid()
plt.legend()
plt.show()
```




```
=== Largest 8 DFT entries by absolute value: ===
(335, 173.92686488373243)
(25, 173.92686488373243)
(26, 25.447354543390354)
(334, 25.44735454339035)
(336, 21.75841585049286)
(24, 21.758415850492856)
(23, 11.642615993684169)
```

```
(337, 11.642615993684165)
```



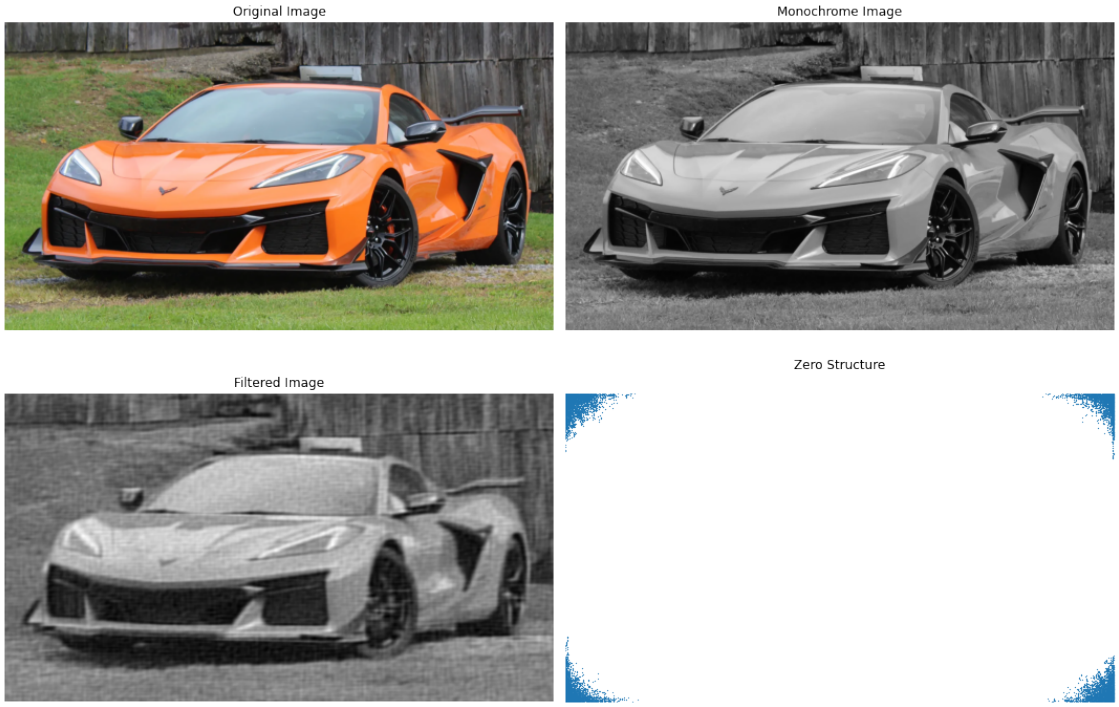
The DFT shows two dominant peaks corresponding to the cosine component of the signal. The smaller coefficients are from the white noise. The largest coefficients are at $k = 25$ and $k = 335$, which is to be expected for a real valued signal (equal magnitude coefficients at k and $360 - k$). The nearby coefficients at $k = 24$ and $k = 26$ may be the result of leakage from sampling. The inverse DFT of the zeroed out DFT produces a reconstructed signal that closely approximates the original. Much of the noise is gone, indicating the original is captured by a few coefficients.

Q12

[7]:
```
# set path to image
img_path = "C:/Users/Ram Prasad/Downloads/C8_Z06.png"

# import as numpy array
c8 = plt.imread(img_path)
print(f"Image array dimensions: {c8.shape}")

# convert to monochrome (exclude alpha channel)
c8_mc = np.sum(c8[:, :, :3], axis=2) / 3

# take 2D DFT
c8_dft = np.fft.fft2(c8_mc)
```

```python
# zero out entries using 99th percentile tolerance
tol = np.percentile(np.abs(c8_dft),99.0)
print(f"Tolerance used: {tol}")
zero_c8_dft = c8_dft.copy()
zero_c8_dft[np.abs(zero_c8_dft) < tol] = 0.0
pct_zeroed = 100 * np.mean(np.abs(c8_dft) < tol)
print(f"Percent of entries set to zero: {np.round(pct_zeroed,5)}%")

# invert modified DFT
zero_c8 = np.fft.ifft2(zero_c8_dft).real

# display original
fig, axs =  plt.subplots(2,2,figsize=(15,10))
axs[0][0].imshow(c8)
axs[0][0].set_title("Original Image")


# monochrome and filtered image
axs[0][1].imshow(c8_mc, cmap="gray")
axs[0][1].set_title("Monochrome Image")
axs[1][0].imshow(zero_c8, cmap="gray")
axs[1][0].set_title("Filtered Image")

# zero structure of filtered DFT
axs[1][1].spy(zero_c8_dft, markersize=0.3)
axs[1][1].set_title("Zero Structure")

# turn off axes
for row in axs:
    for ax in row:
        ax.axis("off")
plt.tight_layout()
plt.show()
```

```
Image array dimensions: (658, 1174, 4)
Tolerance used: 300.91274148395195
Percent of entries set to zero: 98.99999%
```



I use the 99th percentile of absolute DFT entries as a tolerance, zeroing out all entries with absolute value below that threshold to leave 1% of entries untouched. The top left image is the original color, the top right is monochrome. The bottom left is the filtered image obtained with an inverse DFT on the zeroed-out array. The bottom right is the zero structure of the filtered DFT.